Progress Report: Arabic RAG Query Enhancement

Project Title: Improving Retrieval Recall in Arabic RAG Systems via Query Enhancement
Students: Mohammed Elhaj, Osman Bashir
Supervisor: Dr. Tahani
Reporting Period: January 6 - January 31, 2026
Report Date: January 31, 2026

Executive Summary

Over the past month, we successfully completed **Phase 1 (Baseline Implementation)** of our graduation project. We established reproducible baselines for both sparse (BM25) and dense (mDPR) retrieval on the MIRACL Arabic dataset, conducted comprehensive error analysis, and selected our first query enhancement technique based on quantitative evidence.

Key Achievements:

- ✅ Reproduced MIRACL baselines with exact accuracy (mDPR: 100% match to 3 decimals)
- ✅ Completed quantitative error analysis (N=2,896 queries)
- ✅ Selected Query Expansion technique based on evidence
- ✅ Overcame significant technical challenges (Java conflicts, GPU optimization)
- ✅ Built modular, reusable codebase with clean separation of concerns

Current Status: Ready to begin Phase 2 (Query Enhancement Implementation)

Accomplished Tasks Summary

Phase 1: Baseline Implementation (Completed)

1. ✅ **Task 1.1:** Research Embedding Model Options
2. ✅ **Task 1.2:** Download MIRACL Arabic Dataset
3. ✅ **Task 1.4:** Implement BM25 Baseline Retriever (BM25S)
4. ✅ **Task 2.1:** Decide on Embedding Model (Pyserini pre-built indexes)
5. ✅ **Task 2.2:** Implement Dense Baseline Retriever (mDPR)
6. ✅ **Task 2.3:** Run BM25 Baseline Experiments (Exp 002)
7. ✅ **Task 3.1:** Run Dense Baseline Experiments (Exp 001)
8. ✅ **Task 3.3:** Analyze Baseline Errors (Quantitative + Qualitative)
9. ✅ **Task 3.4:** Select First Query Enhancement Technique

Phase 2: Query Enhancement (In Progress)

10. 🔄 **Task 4.1:** Implement Query Expansion with Normalization (Current)

---

1. Project Overview

1.1 Research Question

How can query enhancement techniques improve retrieval performance in Arabic RAG systems?

1.2 Approach

We adopted a **technology-oriented approach**: implement query enhancement techniques on a baseline RAG system and measure which problems they solve.

Architecture:

```plaintext
User Query → [Query Enhancement Layer] → [Retriever] → Retrieved Passages
```

1.3 Dataset

MIRACL Arabic (Dev Set)

- 2,896 queries
- 2,061,414 Wikipedia passages (MSA)
- 29,197 relevance judgments
- Industry-standard benchmark for Arabic retrieval

1.4 Evaluation Metrics

- **Recall@10:** Proportion of relevant documents in top 10
- **Recall@100:** Proportion of relevant documents in top 100
- **NDCG@10:** Ranking quality in top 10
- **MRR:** Mean Reciprocal Rank

---

2. Completed Work (Phase 1)

2.1 Embedding Model Research (Task 1.1)

Owner: Mohammed Elhaj
Duration: January 6-9, 2026
Status: ✅ Complete

Objective: Research and select embedding model for dense retrieval baseline.

Research Conducted:

- Analyzed 3 state-of-the-art multilingual embedding models:
  - **BGE-M3** (BAAI): 80.2 nDCG@10 on MIRACL Arabic
  - **Multilingual E5** (Microsoft): 76.0 nDCG@10
  - **Jina-v3** (Jina AI): Not evaluated on MIRACL
- Reviewed full academic papers for each model
- Compared performance, cost, and implementation complexity
- Discovered Pyserini pre-built indexes for MIRACL

Decision: Use Pyserini pre-built indexes (BM25 + mDPR) for initial baselines

- **Rationale:** Fastest path to implement Query Enhancement; mDPR intentionally "weaker" (not fine-tuned on MIRACL) provides more room for improvement
- **Benefit:** Avoids 12-15 hour embedding time on Google Colab

Documentation: `research_decisions/embedding_model_research.md`

2.2 BM25 Baseline Implementation (Task 1.4)

Owner: Osman Bashir
Duration: January 10-26, 2026
Status: ✅ Complete

Objective: Implement sparse retrieval baseline using BM25.

Challenge: Pyserini requires Java 11, but system had Java 21. Official MIRACL index built with Lucene 8 (Java 11) is incompatible with Java 21's Arabic analyzer.

Solution: Switched to **BM25S** (pure Python implementation)

- No Java dependencies
- Clean API for query enhancement integration
- Achieves 96% of Pyserini performance

Results:

Metric	BM25S Result	MIRACL Target	Achievement
Recall@100	0.8577	0.889	96.5%
NDCG@10	0.4621	0.481	96.0%
Recall@10	0.5964	-	(Baseline)
MRR	0.4836	-	-

Technical Details:

- Library: BM25S v0.2+
- Tokenization: Arabic stemming (PyStemmer) + NLTK stopwords
- Parameters: k1=0.9, b=0.4 (Lucene-style)
- Runtime: ~5 minutes for 2,896 queries

Documentation: `reports/bm25_baseline_report.md` , `docs/experiments/exp_002_baseline_bm25.md`

2.3 Dense Baseline Implementation (Task 2.2)

Owner: Osman Bashir
Duration: January 14-16, 2026
Status: ✅ Complete

Objective: Implement dense retrieval baseline using mDPR.

Challenge: Pyserini's default query encoding is CPU-based and sequential (35 minutes for 2,896 queries).

Solution: GPU-accelerated batch encoding

- Manually encode queries on GPU (64 queries/batch)
- Bypass Pyserini's slow CPU encoder
- Achieve 5-7x speedup (2-3 minutes vs 35 minutes)

Results:

Metric	mDPR Result	MIRACL Target	Match
Recall@100	0.8407	0.841	✅ Exact (3 decimals)
NDCG@10	0.4993	0.499	✅ Exact (3 decimals)
Recall@10	0.6156	-	(Baseline)
MRR	0.5328	-	-

Note: MIRACL targets are reported to 3 decimal places (0.841, 0.499). Our results match exactly when rounded to 3 decimals, confirming successful reproduction.

Technical Details:

- Model: castorini/mdpr-tied-pft-msmarco
- Index: Pyserini pre-built FAISS (5.47 GB)
- Hardware: Google Colab T4 GPU
- Runtime: 2-3 minutes for 2,896 queries
- GPU utilization: 80-90% during encoding

Documentation: `reports/mdpr_baseline_report.md` , `docs/experiments/exp_001_baseline_dense.md`

2.4 Baseline Comparison

Comparison of BM25 vs mDPR:

Metric	mDPR (Dense)	BM25S (Sparse)	Winner
Recall@100	0.8407	0.8577	BM25 (+2.0%)
NDCG@10	0.4993	0.4621	mDPR (+8.1%)
Recall@10	0.6156	0.5964	mDPR (+3.2%)
MRR	0.5328	0.4836	mDPR (+10.2%)

Key Insight: BM25 retrieves more relevant documents (higher Recall@100), but mDPR ranks them better (higher NDCG@10, MRR). This suggests complementary strengths and potential for hybrid approaches.

2.5 Error Analysis (Task 3.3)

Owners: Mohammed Elhaj, Osman Bashir
Duration: January 17, 2026
Status: ✅ Complete

Objective: Understand which queries fail and why, to inform query enhancement technique selection.

Phase 1: Quantitative Analysis (N=2,896 queries)

Key Findings:

1. **39% failure rate** (1,130 queries with NDCG@10 < 0.3)
2. **Short query performance gap:**

    - Short queries (1-3 tokens): NDCG = 0.240
    - Long queries (9+ tokens): NDCG = 0.406
    - **Short queries achieve only 59% of long query performance**
3. **Query length correlation:** r=0.125 (p<0.001, weak but significant)
4. **Retrieval vs ranking gap:** 84% Recall@100 but 50% NDCG@10

Primary Insight: Information poverty in short queries is a validated, dataset-wide driver of failure.

Phase 2: Qualitative Analysis (N=20 sample)

- Observed spelling variations, entity mismatches, diacritics in sample
- Status: Exploratory hypotheses only, NOT used for decision-making
- Confidence: Low (±21% CI on percentages)

Scientific Validation:

- Expert review by Gemini (Antigravity AI): ✅ Approved
- Decision basis: Quantitative evidence only (short query gap)
- Methodology validated as scientifically sound

Documentation: `ERROR_ANALYSIS_COMPLETE.md` , `research_decisions/error_analysis_phase1_quantitative.md`

2.6 Query Enhancement Technique Selection (Task 3.4)

Owners: Mohammed Elhaj, Osman Bashir
Duration: January 17, 2026
Status: ✅ Complete

Objective: Select first query enhancement technique based on error analysis.

Decision: Query Expansion with Normalization

Justification (Evidence-Based):

- **Problem identified:** Short queries lack information/context (proven, N=2,896)
- **Solution:** Query Expansion systematically adds context to address information poverty
- **Secondary:** Normalization as low-cost preprocessing for potential spelling issues

Implementation Approach:

1. **Normalization:** Fix spelling, remove diacritics, standardize spacing
2. **Expansion:** Use Gemini 1.5 Flash to add synonyms, entity variants, related terms

Hypothesis to Test: Query Expansion will improve performance by addressing short query information poverty. Actual impact will be measured in Experiment 003.

Alternative: HyDE (if expansion shows <15% improvement)

Documentation: `research_decisions/qe_technique_selection.md`

---

3. Code Architecture & Modular Structure

We designed a clean, modular codebase with separation of concerns to enable rapid experimentation and easy integration of query enhancement techniques.

3.1 Project Structure

```
plaintext
```

```
arabic-rag-query-enhancement/
├── src/                          # Source code (modular components)
│   ├── retrievers/              # Retrieval implementations
│   │   ├── bm25.py              # BM25S sparse retriever
│   │   └── dense.py             # mDPR dense retriever
│   ├── enhancers/               # Query enhancement modules
│   │   ├── base.py              # Base classes (QueryEnhancer, IdentityEnhancer)
│   │   └── [expansion.py]       # Query expansion (planned)
│   ├── evaluation/              # Evaluation utilities
│   │   └── metrics.py           # Metric computation (Recall, NDCG, MRR)
│   ├── utils/                   # Utility functions
│   │   ├── data_loader.py       # Pyserini data loading
│   │   └── data_loader_hf.py    # HuggingFace data loading
│   └── analysis/                # Analysis scripts
│       ├── load_exp001_data.py
│       └── analyze_exp001_quantitative.py
├── experiments/                  # Jupyter notebooks
│   ├── exp_001_baseline_dense.ipynb
│   └── exp_002_baseline_bm25.ipynb
├── results/                     # Experiment results (TREC format)
│   ├── baseline_dense/
│   └── baseline_bm25/
├── docs/experiments/            # Experiment documentation
│   ├── exp_001_baseline_dense.md
│   └── exp_002_baseline_bm25.md
└── requirements.txt             # Python dependencies
```

3.2 Design Principles

1. Separation of Concerns:

- **Retrievers:** Handle document retrieval (BM25, Dense)

- **Enhancers:** Transform queries before retrieval
- **Evaluation:** Compute metrics independently
- **Utils:** Shared utilities (data loading, preprocessing)

2. Modular Components:

- Each module can be used independently
- Easy to swap implementations (e.g., BM25S vs Pyserini)
- Clean interfaces for integration

3. Extensibility:

- Base classes for query enhancers (`QueryEnhancer`)
- Easy to add new enhancement techniques
- Plug-and-play architecture

4. Reproducibility:

- All experiments documented in `docs/experiments/`
- Results saved in standard TREC format
- Configuration files for hyperparameters

3.3 Benefits of Modular Design

1. **Rapid Experimentation:** Swap components without rewriting code
2. **Code Reuse:** Shared utilities across experiments
3. **Easy Testing:** Test components independently
4. **Clear Documentation:** Each module has clear responsibility
5. **Collaboration:** Mohammed and Osman can work on different modules in parallel

---

4. Technical Challenges & Solutions

4.1 Challenge: Java Version Conflict (BM25)

Problem: Pyserini requires Java 11, but system had Java 21. Official MIRACL index incompatible with Java 21's Arabic analyzer.

Attempted Solutions:

1. Downgrade Java (failed - system conflicts)
2. Rebuild index with Java 21 (79% performance vs 89% target)
3. Use Conda environment with Java 11 (CLI worked, Python code failed)

Final Solution: Switch to BM25S (pure Python)

- **Trade-off:** 96% of Pyserini performance (vs 100%)

- **Benefits:** No Java dependencies, easier QE integration, pure Python
- **Conclusion:** Acceptable trade-off for thesis goals

Lesson Learned: Pure Python implementations provide better reproducibility and integration for our use case.

4.2 Challenge: Slow Query Encoding (mDPR)

Problem: Pyserini's default query encoding is CPU-based and sequential (35 minutes for 2,896 queries).

Solution: GPU-accelerated batch encoding

- Manually encode queries on GPU (64 queries/batch)
- Bypass Pyserini's slow CPU encoder
- Achieve 5-7x speedup (2-3 minutes vs 35 minutes)

Implementation:

```python
# Tokenize and move to GPU
inputs = tokenizer(batch, ...).to(device)

# Encode on GPU
outputs = model(**inputs)
embeddings = outputs.last_hidden_state[:, 0, :]

# Normalize for cosine similarity
embeddings = torch.nn.functional.normalize(embeddings, p=2, dim=1)
```

Lesson Learned: GPU acceleration is essential for rapid experimentation. Custom implementations can significantly outperform default tools.

4.3 Challenge: Error Analysis Without Metadata

Problem: MIRACL passages lack metadata (no domain labels like Law, Medical, etc.) for categorization.

Research Conducted:

- Confirmed by 4 research providers (Gemini, Perplexity, Context7, Langfuse)
- Explored alternatives: NoMIRACL hard negatives, Wikipedia categories, AAFAQ taxonomy

Solution: Focus on query-side analysis

- Query length correlation (highest ROI)
- Score gap analysis
- Rank distribution analysis
- Future: Wikipedia categories via API, NoMIRACL hard negatives

Lesson Learned: When dataset lacks metadata, focus on query-side analysis provides highest ROI for limited timeline.

---

5. Current Work (Week 5)

5.1 Mohammed Elhaj: LLM Research for Query Enhancement

Objective: Research models for query enhancement layer based on:

- Model size (resource constraints)
- Arabic capability (MSA quality)
- Quantization ability (deployment efficiency)
- Cost effectiveness (API budget)

Candidates:

- Gemini 1.5 Flash (current choice - free tier, fast)
- GPT-4o-mini (backup - low cost)
- Qwen 2.5 (3B, 7B variants - local deployment)
- Jais (Arabic-specific)

Status: In progress

5.2 Osman Bashir: Qwen 2.5 3B Testing

Objective: Test and tune Qwen 2.5 3B model for query expansion using Query2Doc paper approach.

Approach:

- Implement Query2Doc prompt engineering
- Test on sample queries from error analysis
- Measure expansion quality
- Compare with Gemini 1.5 Flash baseline

Status: In progress

---

6. Timeline & Next Steps

Completed (Weeks 1-4)

- ✅ Week 1: Embedding model research, dataset setup
- ✅ Week 2: BM25 baseline implementation (with Java conflict resolution)
- ✅ Week 3: Dense baseline implementation, error analysis
- ✅ Week 4: Error analysis completion, technique selection

Upcoming (Weeks 5-6)

- **Week 5 (Current):**
  - LLM research for query enhancement (Mohammed)
  - Qwen 2.5 3B testing (Osman)
  - Implement Query Expansion with Normalization (Task 4.1)
- **Week 6:**
  - Run Experiment 003 (QE + Dense)
  - Run Experiment 004 (QE + BM25)
  - Analyze results and iterate if needed
  - Begin thesis documentation

Deadline: February 15, 2026 (2 weeks remaining)

---

7. Key Deliverables

7.1 Code & Implementation

- ✅ BM25 baseline retriever (`src/retrievers/bm25.py`)
- ✅ mDPR baseline retriever (`src/retrievers/dense.py`)
- ✅ Evaluation pipeline (`src/evaluation/metrics.py`)
- ✅ Data loader utilities (`src/utils/data_loader.py` , `src/utils/data_loader_hf.py`)
- ✅ Base query enhancer classes (`src/enhancers/base.py`)
- ⏳ Query expansion module (in progress)

7.2 Documentation

- ✅ Embedding model research report
- ✅ BM25 technical report (5 implementation attempts documented)
- ✅ mDPR technical report (GPU optimization documented)
- ✅ Error analysis reports (quantitative + qualitative)
- ✅ Experiment documentation (Exp 001, Exp 002)
- ✅ Scientific review validation

7.3 Experiment Results

- ✅ Experiment 001: Dense Baseline (mDPR + Identity Enhancement)
- ✅ Experiment 002: Sparse Baseline (BM25S + Identity Enhancement)
- ⏳ Experiment 003: Query Expansion + Dense (planned)
- ⏳ Experiment 004: Query Expansion + Sparse (planned)

---

8. Lessons Learned

8.1 Technical Lessons

1. **Pure Python > Java dependencies:** BM25S provides better reproducibility than Pyserini
2. **GPU acceleration is essential:** 5-7x speedup enables rapid experimentation
3. **Modular design pays off:** Easy to swap query enhancers, clean separation of concerns
4. **Pre-built indexes save time:** Avoided 12-15 hours of embedding time

8.2 Research Lessons

1. **Quantitative evidence > qualitative observations:** Small sample qualitative analysis (N=20) has low confidence (±21% CI)
2. **Query-side analysis is highest ROI:** When dataset lacks metadata, focus on query characteristics
3. **Complementary strengths:** BM25 better at recall, mDPR better at ranking - suggests hybrid potential
4. **Intentionally weak baseline:** mDPR (not fine-tuned on MIRACL) provides more room to demonstrate QE improvement

8.3 Project Management Lessons

1. **Document challenges:** Technical blockers (Java conflicts) became valuable learning experiences
2. **Validate decisions:** Scientific review (Gemini expert) validated our methodology
3. **Iterate quickly:** GPU optimization and pure Python approach enabled faster iteration
4. **Focus on evidence:** Base decisions on quantitative data, not assumptions

---

9. Risks & Mitigation

9.1 Current Risks

1. **Timeline pressure:** 2 weeks remaining for implementation, experiments, and documentation

    - **Mitigation:** Focus on single QE technique (Query Expansion), defer alternatives
2. **LLM API costs:** Gemini free tier has rate limits (15 RPM)
    - **Mitigation:** Batch queries, cache expansions, fallback to local LLM (Qwen 2.5 3B)
3. **Query Expansion may not improve:** No guaranteed ROI
    - **Mitigation:** HyDE as backup technique, hybrid approaches as alternative

9.2 Mitigated Risks

- ✅ **Java conflicts:** Resolved via BM25S
- ✅ **Slow encoding:** Resolved via GPU acceleration
- ✅ **Metadata absence:** Resolved via query-side analysis
- ✅ **Embedding time:** Resolved via Pyserini pre-built indexes

---

10. Publications & References

10.1 Papers Reviewed

1. **MIRACL Dataset:** Zhang et al. (2022) - Multilingual retrieval benchmark
2. **BGE-M3:** Chen et al. (2024) - Multi-functional multilingual embeddings
3. **Multilingual E5:** Wang et al. (2024) - Multilingual text embeddings
4. **Jina-v3:** Sturua et al. (2024) - Task-specific embeddings
5. **HyDE:** Gao et al. (2022) - Hypothetical document embeddings
6. **Query2Doc:** Wang et al. (2023) - Query expansion with LLMs
7. **QE-RAG:** Chen et al. (2025) - Query enhancement for RAG

10.2 Tools & Libraries

- **Pyserini:** IR toolkit with pre-built indexes
- **BM25S:** Pure Python BM25 implementation
- **HuggingFace Transformers:** Model loading and inference
- **FAISS:** Vector similarity search
- **pytrec-eval:** Standard IR evaluation metrics

---

11. Conclusion

Over the past month, we successfully established a solid foundation for our graduation project. We overcame significant technical challenges (Java conflicts, slow encoding), reproduced MIRACL baselines with exact accuracy, and conducted rigorous error analysis to inform our query enhancement approach.

Key Achievements:

- ✅ Reproducible baselines (BM25: 0.8577 Recall@100, mDPR: 0.8407 Recall@100)
- ✅ Evidence-based technique selection (Query Expansion for short query gap)
- ✅ Scientific validation (Gemini expert review approved)
- ✅ Modular codebase ready for Phase 2 implementation

Current Focus:

- Mohammed: LLM research for query enhancement layer
- Osman: Qwen 2.5 3B testing for query expansion

Next Milestone: Experiment 003 (Query Expansion + Dense) - Target completion: February 7, 2026

We are confident in our progress and methodology, and we look forward to demonstrating measurable improvements in Phase 2.

Prepared by: Mohammed Elhaj, Osman Bashir
Date: January 31, 2026
Supervisor: Dr. Tahani
Institution: University of Khartoum, Department of Electrical and Electronic Engineering